
Byzantine Consensus

Mohsen Lesani

(Strong) Byzantine Consensus

Module:

- ByzantineConsensus, instance bc.

Events:

- Request: $\langle \text{propose}(v) \rangle$:
Proposes value v .
- Indication: $\langle \text{decide}(v) \rangle$:
Outputs the decided value v .

(Strong) Byzantine Consensus

Properties

- WBC1: *Termination*:
Every correct process eventually decides some value.
- **BC2: *Strong Validity***:
A correct process may decide only a value that was proposed by some **correct** process or the special value $\square$. Further, if all correct processes propose the same value v , then no correct process decides a value different from v .
- WBC3: *Integrity*:
No correct process decides twice.
- WBC4: *Agreement*:
No two correct processes decide differently.

The only way to know that a value is from a correct process is that we show that it is from $f+1$ processes. That's why when we cannot, we have to decide the special $\square$ value.

The total order broadcast abstraction that uses consensus has an initial round where processes exchange their messages so that they propose the same value.

Weak Byzantine Consensus

Module:

- WeakByzantineConsensus, instance wbc.

Events:

- Request: $\langle \text{propose}(v) \rangle$:
Proposes value v .
- Indication: $\langle \text{decide}(v) \rangle$:
Outputs a decided value v .

Weak Byzantine Consensus

Properties

- WBC1: *Termination*:
Every correct process eventually decides some value.
- **WBC2: *Weak validity***:
A correct process may only decide a value that was proposed by **some process**.
- WBC3: *Integrity*:
No correct process decides twice.
- WBC4: *Agreement*:
No two correct processes decide differently.

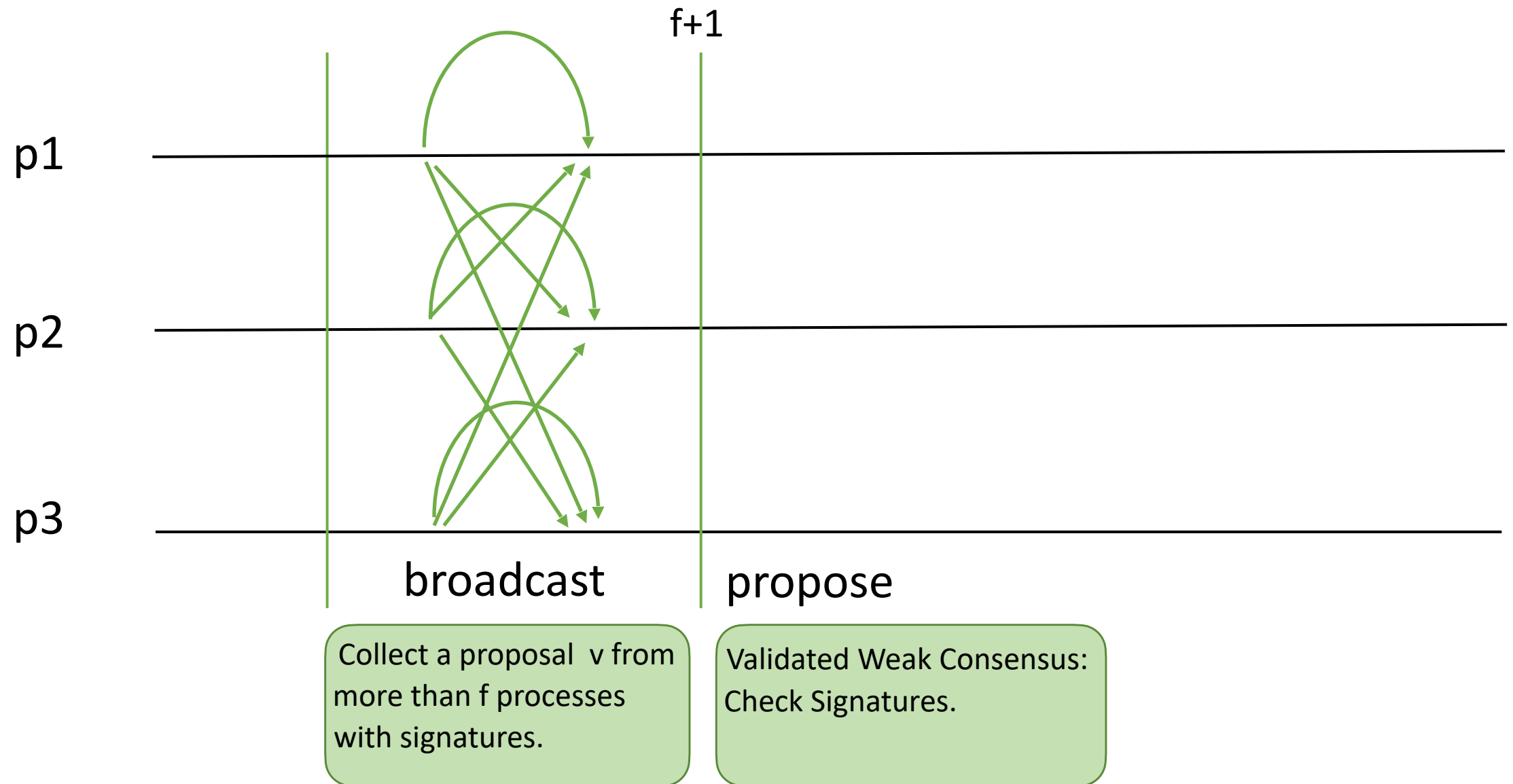
From Weak to Strong Consensus

We want only a proposal from a correct process to be decided.

Idea:

1. Make sure the proposal is from a correct process:
Collect proposals and **signatures**. Adopt a proposal only if it is received from **more than f processes**. There is at least one correct process in that set.
Propose that proposal together with the signatures. Otherwise, propose the special $\square$ value.
2. Weak consensus can decide a value proposed from a Byzantine process.
Therefore, **validate** decisions from weak consensus: it has either $f+1$ signatures, or it is the $\square$ value.
A **Byzantine process** is forced to behave. If it wants to propose a value that is accepted, it must propose a value that was initially proposed by at least one correct process (or propose $\square$).

From Weak to Strong Consensus



Validating the Proposal

A proposal is valid if it is signed by at least $f+1$ processes or it is the special $\square$ value.

```
function valid-proposal(  $(v, \Sigma)$  )  
    if  $\#(\{p \in \pi \mid \text{verify-sig}(p, v, \Sigma[p]) = \text{true}\}) > f$  then  
        return true;  
    else if  $v = \square$  then  
        return true  
    else  
        return false
```

A predicate on values.

Update the weak consensus protocol so that a process that receives a value that does not satisfy the predicate halts.

From Weak to Strong Consensus

Implements:

ByzantineConsensus, instance bc.

Uses:

AuthPerfectPointToPointLinks, instance al

WeakByzantineConsensus (Validated), instance wc

upon event < init > **do**

proposals := $[\perp]^N$

$\Sigma := [\perp]^N$

From Weak to Strong Consensus

upon event $\langle \text{propose}(v) \rangle$ **do**

$\sigma := \text{sign}(\text{self}, v)$

forall $q \in \pi$ **do**

trigger $\langle \text{al}, \text{send}(q, \text{Proposal}(v, \sigma)) \rangle$

upon event $\langle \text{al}, \text{deliver}(p, \text{Proposal}(v, \sigma)) \rangle$ **do**

if $\text{proposals}[p] = \perp \wedge \text{verify-sig}(p, v, \sigma)$ **then**

$\text{proposals}[p] := v$

$\Sigma[p] := \sigma$

From Weak to Strong Consensus

upon exists $v \neq \perp$ such that $\#(\{p \in \pi \mid \text{proposals}[p] = v\}) > f$ **do**

proposals $:= [\perp]^N$

trigger $< \text{wc}, \text{propose}((v, \Sigma)) >$

upon ($\#\text{proposals} \geq N - f$) **do**

proposals $:= [\perp]^N$

trigger $< \text{wc}, \text{propose}((\square, \Sigma)) >$

upon event $< \text{wc}, \text{decide}((v', \Sigma')) >$ **do**

trigger $< \text{decide}(v') >$

Collect signatures from more than f processes for the same value and propose it.

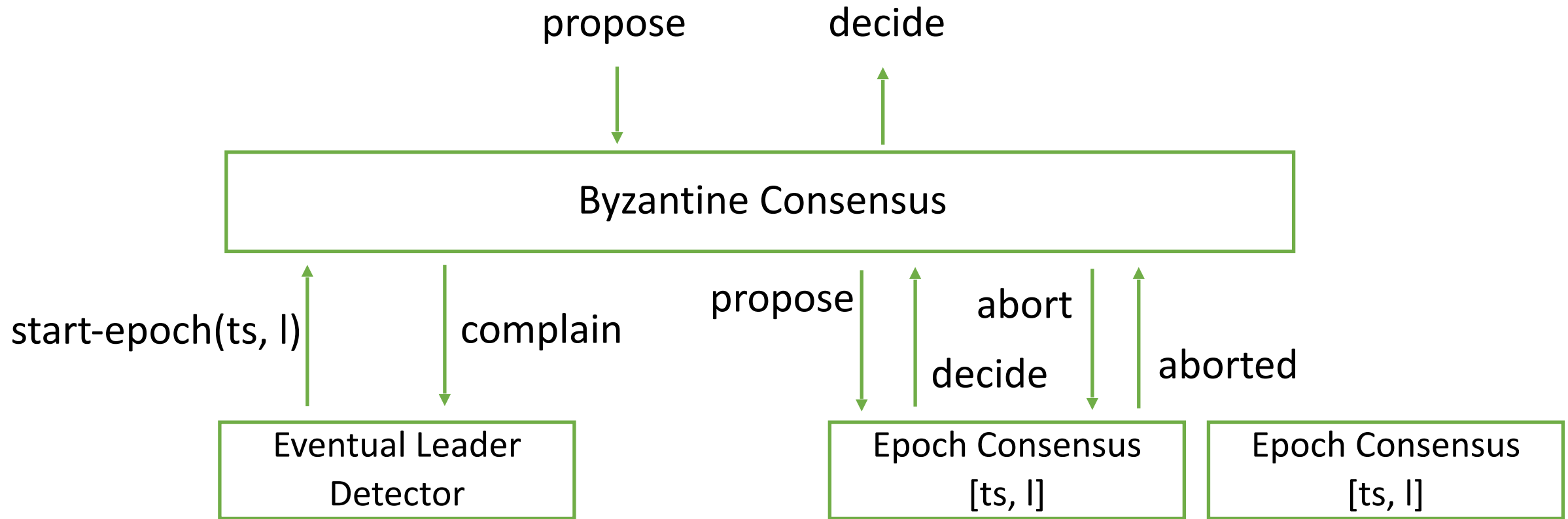
When there is no hope that it happens, propose the special $\square$ value. The remaining f processes may be Byzantine and may never respond.

From Weak to Strong Validity

Validity: A correct process may decide only a value that was proposed by some correct process or the special value $\square$.

- A correct process finally decides only a value that is decided by validated weak consensus.
- Validated weak consensus decides only decides a value that is validated to either come with $f+1$ signatures, or is $\square$.
- If the value comes with $f+1$ signatures, at least one signature is from a correct process. Thus, the value is proposed by a correct process.

Leader-driven Weak Consensus



Similar to the protocol for the fail-noisy environment

Byzantine Eventual Leader Detector

- We cannot rely on the timeliness of **heartbeats** for detecting arbitrary faults. A Byzantine process can be responsive to a process and not the other.
- If the leader performs wrongly or exceeds the allocated time before reaching the goal, then other processes **detect** this and **complain** to their local leader detector component.
- The leader detector component gets input from the higher-level Byzantine consensus component as complain requests.

upon event (timeout)
trigger < bld, complain(l) >

Eventual Byzantine Leader Detector

Module:

ByzantineLeaderDetector, instance bld.

Events:

- **Request:**

< complain (p) >

Receives a complaint about process p.

- **Indication:**

< trust (p) >

Indicates that process p is trusted to be leader.

Eventual Byzantine Leader Detector

Properties:

- **BLD1: *Eventual succession:***
If more than f correct processes (that trust some process p) complain about p , then every correct process eventually trusts a different process than p .
- **BLD2: *Putsch resistance:***
A correct process does not trust a new leader unless at least one correct process has complained against the current leader.
- **BLD3: *Eventual agreement:***
There is a time after which no two correct processes trust different processes.

Eventual Byzantine Leader Detector

- By properties 1 and 3, every correct process eventually trusts some process that appears to perform its task in the high-level component.
- In contrast to the crash model, one cannot require that every correct process eventually trusts a correct process because a Byzantine process may behave just like a correct process.

Rotating Byzantine Leader Detector

Idea:

- Round robin: the round number deterministically derives the leader of the round.
- A process considers a complaint about the current leader seriously only if it receives it from more than f processes. It then broadcasts a complaint itself if it has not already. This amplifies the complain.
- A process moves to the next round and its leader only if it receives the complaint from more than $2f$ processes. This ensures that after it moves to the next round, there are enough correct processes complaining that can push all correct processes to the next round.
- This is similar to the last phase of Bracha Byzantine broadcast: the complain message is like the ready message.
- It is assumed that $n > 3f$.

Protocol: Rotating Byzantine Leader Detector

Implements:

ByzantineLeaderDetector, instance bld.

Uses:

AuthPerfectPointToPointLinks, instance al.

upon event < init > **do**

round := 1

complains := $[\perp]^N$

complained := **false**

trigger < trust(leader(round)) >

upon event < complain(p) > such that

p = leader(round) and complained = **false** **do**

complained := **true**

forall q $\in \pi$ **do**

trigger < al, send(q, Complain(round)) >

A complaint from the higher-level component.

Protocol: Rotating Byzantine leader detector

upon event $\langle a_l, \text{deliver}(p, \text{Complain}(r)) \rangle$ such that
 $r = \text{round}$ and $\text{complains}[p] = \perp$ **do**
 $\text{complains}[p] := \text{true}$
 if $\#(\text{complains}) > f \wedge \text{complained} = \text{false}$ **then**
 $\text{complained} := \text{true}$
 forall $q \in \pi$ **do**
 trigger $\langle a_l, \text{send}(q, \text{Complain}(\text{round})) \rangle$
 else if $\#(\text{complains}) > 2f$ **then**
 $\text{round} := \text{round} + 1$
 $\text{complains} := [\perp]^N$
 $\text{complained} := \text{false}$
 trigger $\langle \text{trust}(\text{leader}(\text{round})) \rangle$

Byzantine Epoch-Change

Module:

- **Name:** ByzantineEpochChange, instance bec.

Events:

- **Indication:** $\langle \text{start-epoch}(ts, l) \rangle$
Starts the epoch identified by timestamp ts with leader l .

Properties:

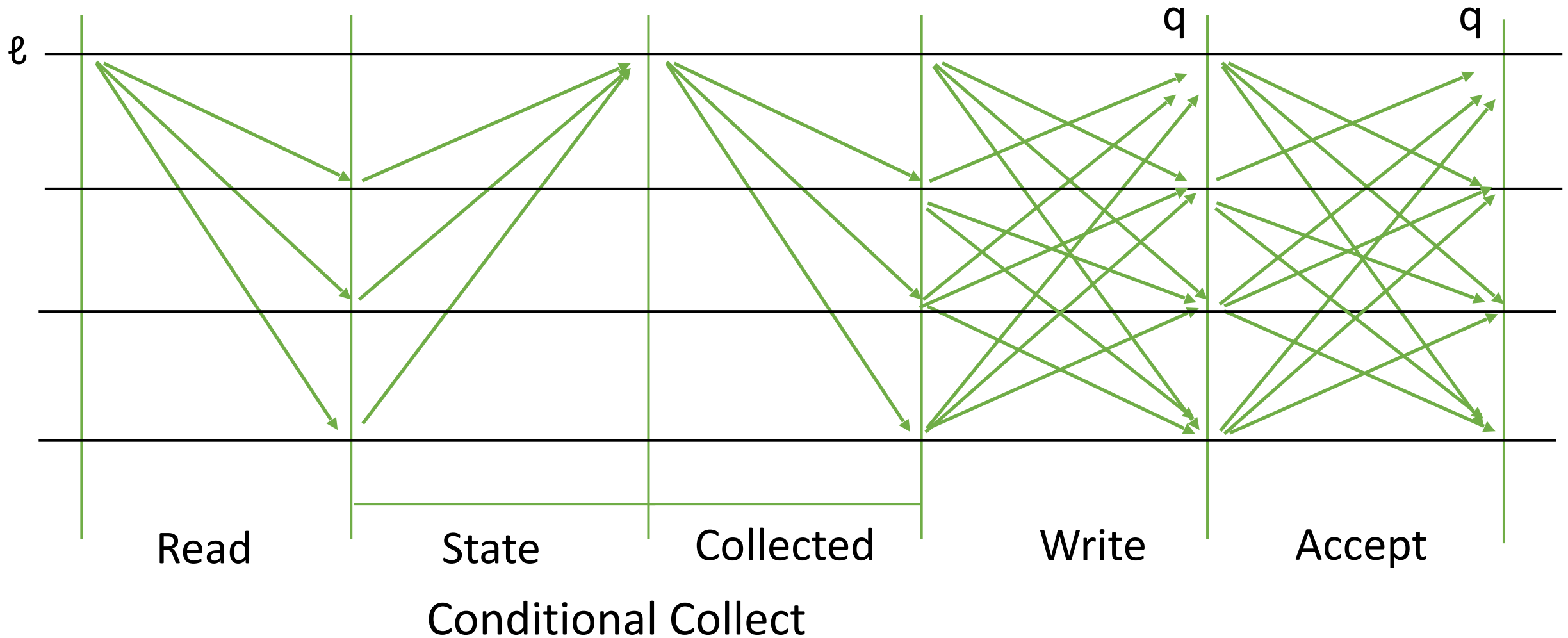
- **EC1: Monotonicity:**
If a correct process starts an epoch (ts, l) and later starts an epoch (ts', l') then $ts' > ts$.
- **EC2: Consistency:**
If a correct process starts an epoch (ts, l) and another correct process starts an epoch (ts, l') then $l = l'$.
- **EC3: Eventual Leadership:**
There is a time after which every correct process has started some epoch (ts, l) and starts no further epoch.

The leader l does not lead to $f+1$ complaints.

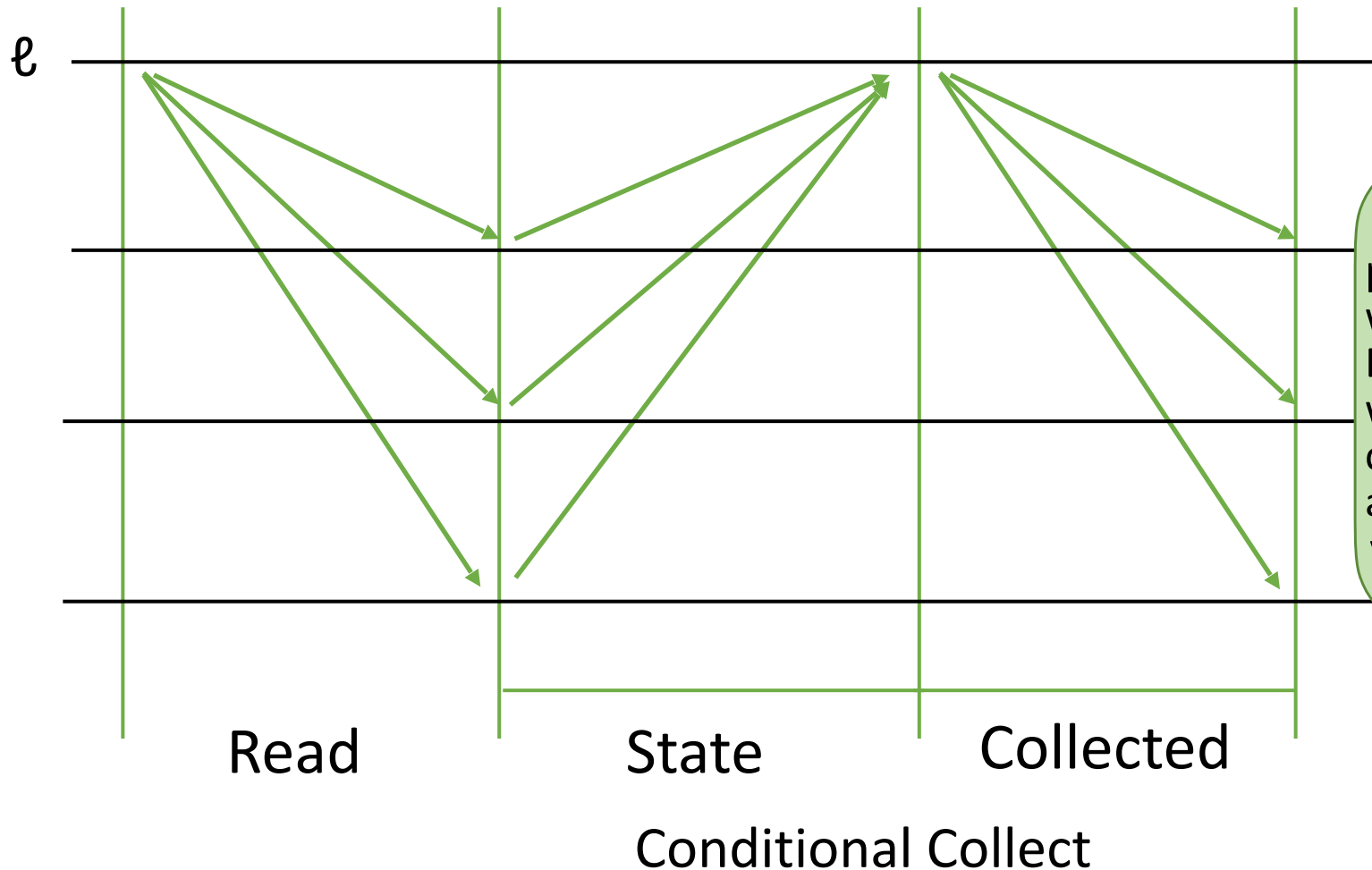
Protocol: Byzantine Epoch-Change

The same as the previous protocol. Just output the round number as the timestamp in addition to the leader.

Byzantine Epoch Consensus



Conditional Collect



The Leader should read the state of processes and determine the value to be written, and send it to all processes to write. But processes cannot trust a single sender with the value that it sends. Thus, the leader collects signed values and forwards them to all processes. They themselves determine the value from the collection.

Conditional Collect

A leader collects messages from others such that the collection satisfies a predicate.

Module:

ConditionalCollect, **instance** cc, with leader / and output predicate C.

Events:

- **Request:**
< input(m) >
Inputs a message m.
- **Indication:**
< collected(M) >:
Outputs a vector M of collected messages.

Protocol: Signed Conditional Collect

Implements:

ConditionalCollect, instance cc, with leader l and output predicate C .

Uses:

AuthPerfectPointToPointLinks, instance al.

upon event < init > **do**

states := $[\perp]^N$; $\Sigma := [\perp]^N$

collected := **false**

upon event < input(s) > **do**

$\sigma := \text{sign}(\text{self}, s)$

trigger < al, send(l , State(s , σ)) >

upon event < al, deliver(p , State(s , σ)) > **do** // only leader l

if verify-sig(p , s , σ) **then**

states[p] := s ; $\Sigma[p]$:= σ

To prevent storing wrong messages in the leader, the signature is checked although the checks are done in followers as well.

Protocol: Signed Conditional Collect

upon $C(\text{states}) \vee \#(\text{states}) \geq N-f$ **do** // only leader /
 forall $q \in \pi$ **do**
 trigger $\langle \text{al}, \text{send}(q, \text{Collected}(\text{states}, \Sigma)) \rangle$
 $\text{states} := [\perp]^N; \Sigma := [\perp]^N$

After waiting for $N-f$, we cannot wait for any more processes because the rest can be Byzantine and may never respond.

upon event $\langle \text{al}, \text{deliver}(l, \text{Collected}(S, \Sigma)) \rangle$ **do**
 if $\text{collected} = \text{false} \wedge C(S) \wedge$
 $\forall p \in \pi. S[p] \neq \perp \Rightarrow \text{verify-sig}(p, S[p], \Sigma[p])$ **then**
 $\text{collected} := \text{true};$
 trigger $\langle \text{collected}(S) \rangle$

Protocol State

State:

(valts, val, cert)

The current timestamp and value pair, and
a certificate that shows they are the result of a valid write.

Collecting the State of a Quorum

Use conditional collect.

The predicate $C(S)$ for Conditional Collect is that

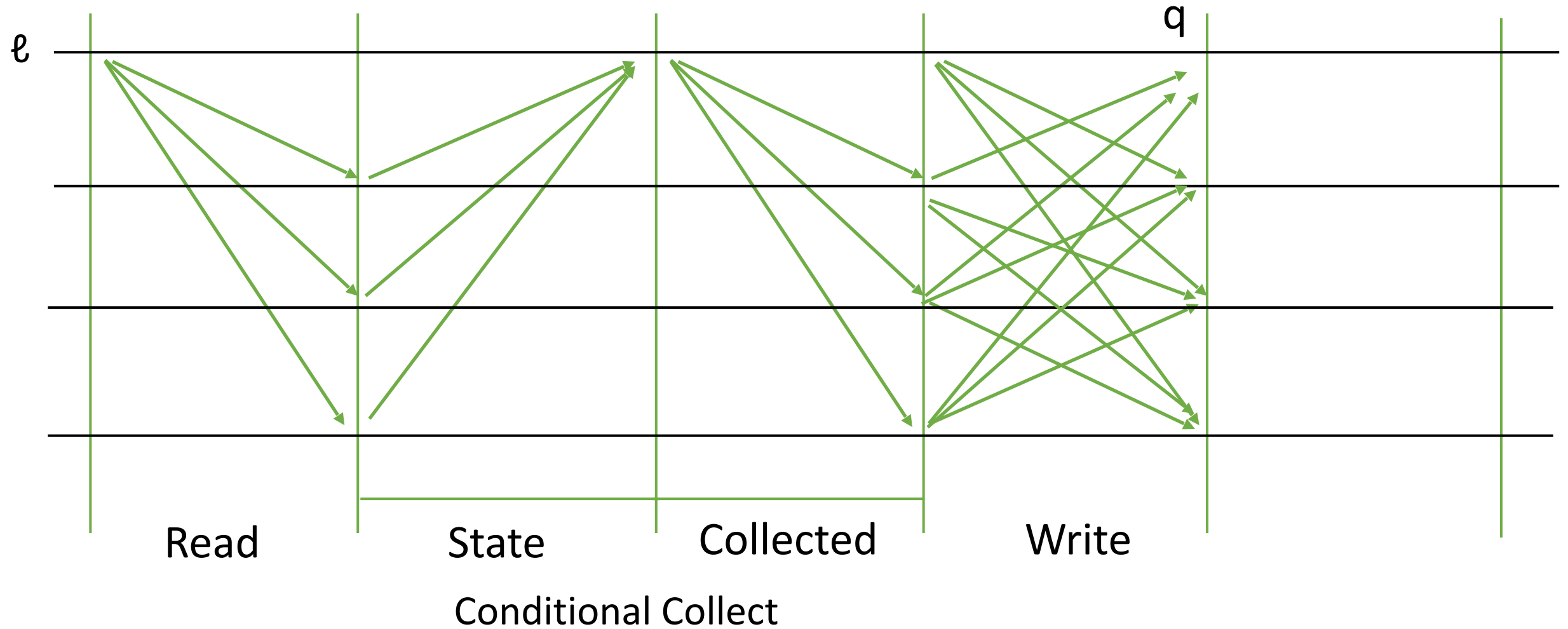
- S is at least a quorum, and
- in each entry of S , cert is valid.

Retrieving Previously Decided Values

With a collection of states S from a Byzantine quorum (more than $2f$ processes) that have valid certs, the consensus is either

- **unbound**, $\text{unbound}(S)$: The largest timestamp is the initial 0.
 - Then, adopt the proposal of leader.
- **bound** to a value v , $\text{binds}(v, S)$: The largest timestamp is positive.
 - Then, the value v of the largest timestamp is already locked-in. Adopt it.

Write Phase



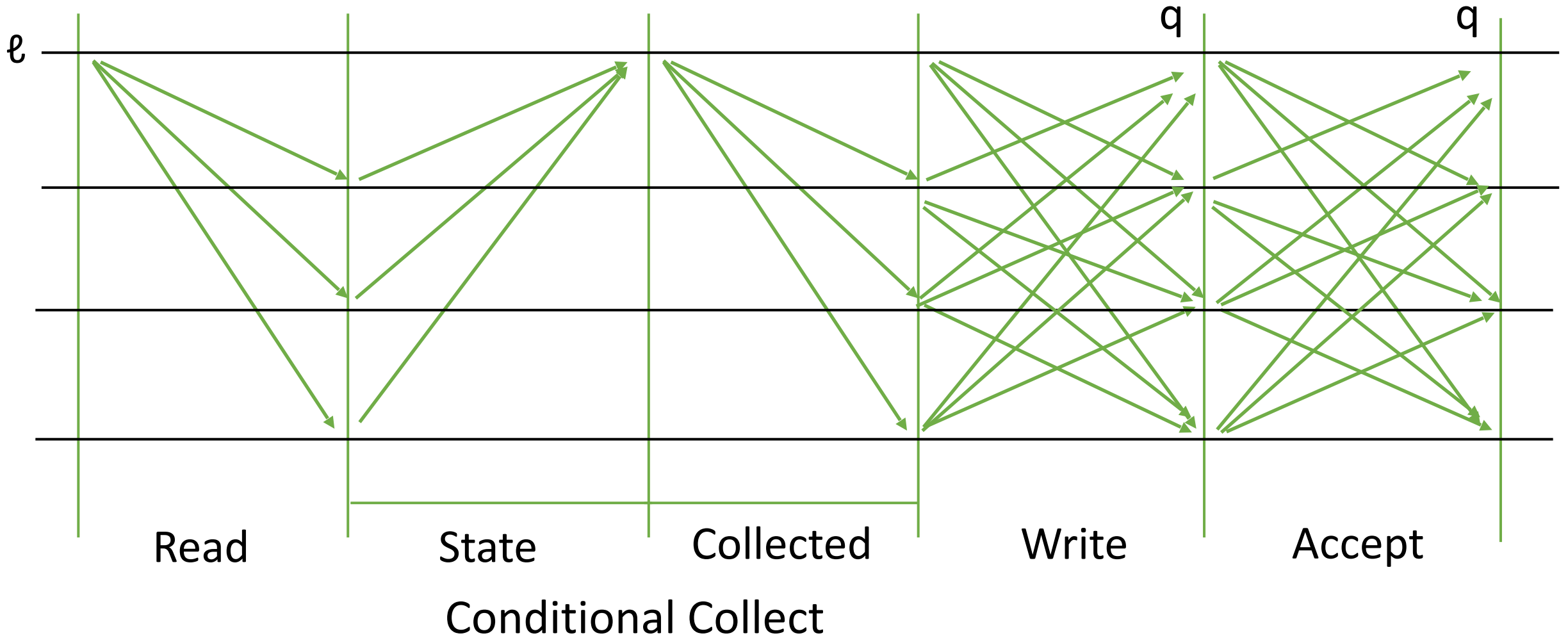
Write Phase

A Byzantine leader may send different values as his own proposal.

Thus, in order to consistently write the same value, processes should receive the same value from a Byzantine quorum.

Thus, similar to Byzantine consistent broadcast, this round needs an all-to-all communication.

Accept Phase



Accept Phase

- The accept round makes sure that before deciding a value, it is already written to a Byzantine quorum.
- It ensures the lock-in property.

Byzantine Epoch Consensus

Module:

ByzantineEpochConsensus, instance bep, with timestamp ts and leader l .

Events:

- **Request:** $\langle \text{propose}(v) \rangle$
Proposes value v . Executed only by the leader l .
- **Indication:** $\langle \text{decide}(v) \rangle$:
Outputs the decided value v .
- **Request:** $\langle \text{abort} \rangle$:
Aborts epoch consensus.
- **Indication:** $\langle \text{aborted}(\text{state}) \rangle$:
Signals that epoch consensus has completed abortion, and outputs the internal state.

Protocol: Byzantine Epoch Consensus

Implements ep,
with timestamp ets and leader L

Uses al, abeb, cc ($N = 3f + 1$)

upon < init (ts, v) > **do**

written := $[\perp]^N$

accepted := $[\perp]^N$

(valts, val) := (ts, v)

cert := $\perp$

upon < propose (v) > **do** ▷ At the leader L

if val = $\perp$ **then** val := v

trigger < abeb, broadcast(Read) >

When val = $\perp$, ts = 0. Thus, the proposal of the leader, val, does not make the collection bound. The value val is passed in the collection to be used when the collection is unbound.

Protocol: Byzantine Epoch Consensus

upon < abeb, deliver(L, Read) > **do**
 trigger <cc, input(State(valts, val, cert))>

upon < cc, collected(S) > **do**
 if $\exists v. \text{binds}(v, S)$ **then**
 $cv := v$
 else if $\text{unbound}(S) \wedge S[L] = \text{State}(_, v, _)$ **then**
 $cv := v$
 trigger <abeb, broadcast(Write(ets, cv))>

For each p:
 $S[p] = \text{State}(ts, v, \text{cert})$ or $\perp$

Protocol: Byzantine Epoch Consensus

upon $\langle \text{abeb}, \text{deliver}(p, \text{Write}(ts, v)^\sigma) \rangle$ where $ts = ets$ **do**
 $\text{written}[p] := v^\sigma$
 if $\exists v. \#\{p \mid \text{written}[p] = v\} \geq 2f+1$ **then**
 $(\text{valts}, \text{val}) := (ets, v)$
 $\text{cert} := \text{written}$
 $\text{written} := [\perp]^N$
 trigger $\langle \text{abeb}, \text{broadcast}(\text{Accept}(v)) \rangle$

Superscript is the signature.

Protocol: Byzantine Epoch Consensus

```
upon < abeb, deliver(p, Accept(v)σ) > do  
  accepted[p] := vσ  
  if  $\exists v. \#\{p \mid \text{accepted}[p] = v\} \geq f+1$  then  
    (valts, val) := (ets, v)  
    cert := accepted  
    trigger <abeb, broadcast(Accept(v))>  
  if  $\exists v. \#\{p \mid \text{accepted}[p] = v\} \geq 2f+1$  then  
    accepted :=  $[\perp]^N$   
    trigger < decide(v) >
```

```
upon < abort > do  
  trigger < aborted(valts, val, cert) >
```

Amplification

Amplification and Termination

Without amplification, consider the following scenario:

The leader is Byzantine. The f byzantine processes communicate in the Write and Accept rounds with only $f+1$ out of $2f+1$ correct processes. The communicating processes $f + f + 1 = 2f + 1$ are enough to make each other decide. However, the remaining f correct processes are left undecided. Since their number is less than f , their complaints cannot start a new epoch.

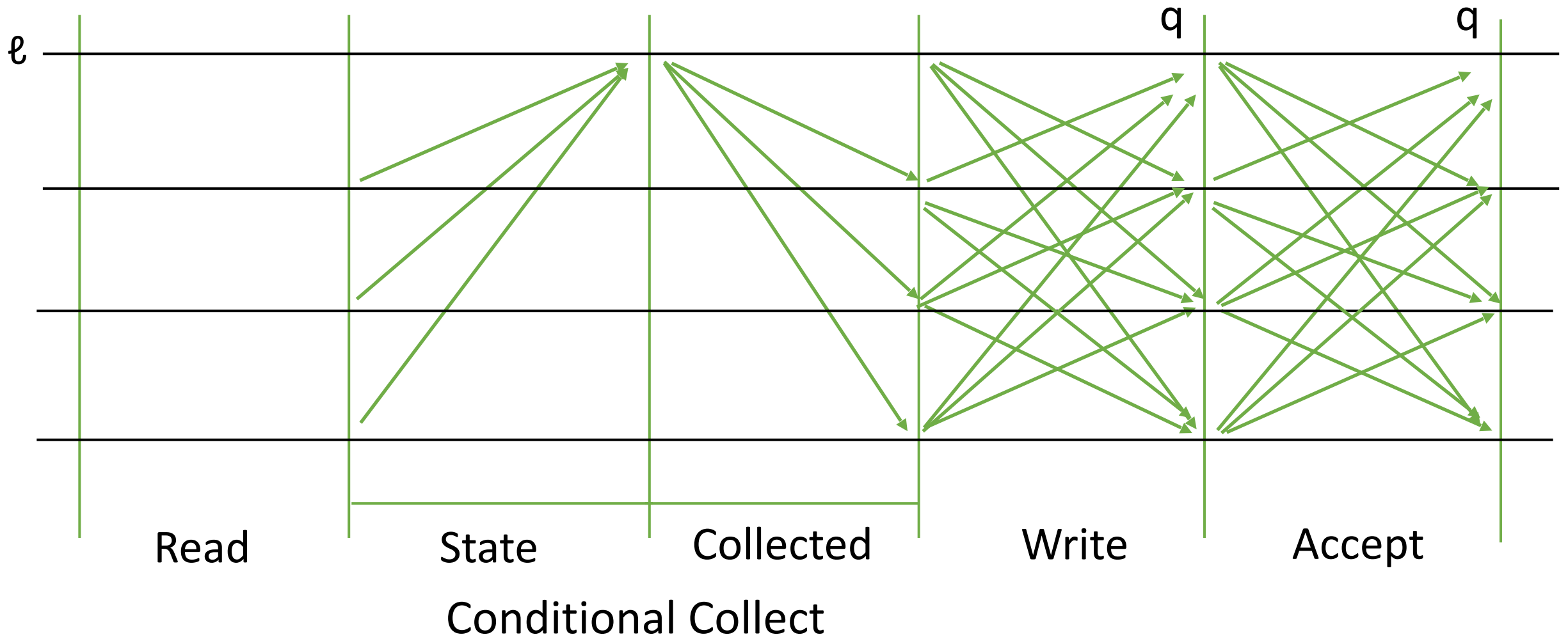
With amplification,

The remaining f correct processes receive $f+1$ Accept messages, and send Accept themselves. Thus, each of them receives $2f+1$ Accept messages, and decides.

Optimizations

First, the Read message may be omitted. Upon initializing the new epoch consensus instance, every process already knows the leader, and simply invokes conditional collect with its State message.

Accept Phase

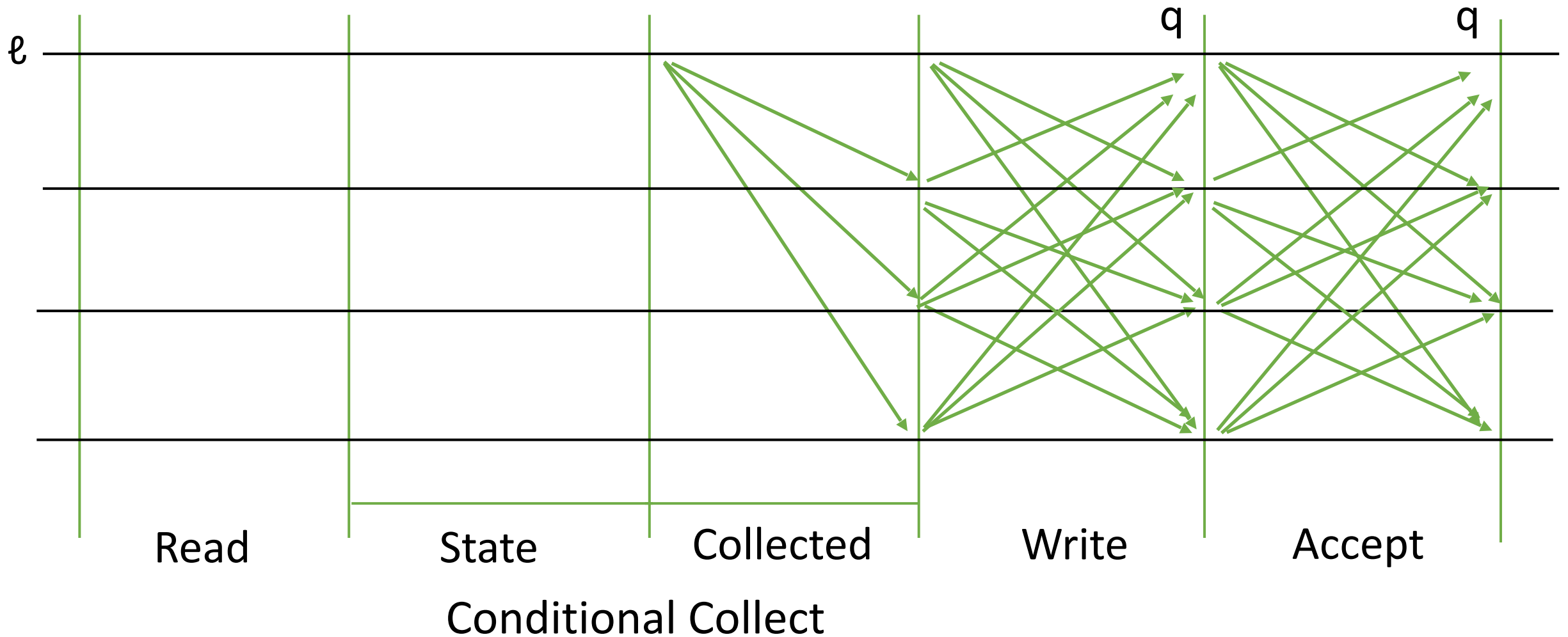


Optimizations

Second, in the first epoch consensus instance, the conditional collect primitive for reading the state of all processes may be skipped because all processes store the default state initially.

Therefore, the algorithm involves an initial message from the leader to all processes and two rounds of echoing the message among all processes. This is the same communication pattern as first used in the Byzantine reliable broadcast algorithm (Bracha), and it is also used during the normal-case operation of a view in the PBFT algorithm.

Accept Phase



Byzantine epoch consensus

Properties:

- **EP1: *Validity*:**
Any value decided by a correct process was proposed by the leader of the current or a previous epoch consensus.
- **EP2: *Agreement*:**
No two correct processes decide differently.
- **EP3: *Integrity*:**
Every correct process decides at most once.
- **EP4: *Lock-in*:**
If a correct process decides a value in an epoch, no correct process decides a different value in future epochs.
- **EP5: *Termination*:**
If the leader is correct, has proposed a value, and no correct process aborts this epoch consensus, then every correct process eventually decides some value.
- **EP6: *Abort behavior*:**
When a correct process requests abort, it will eventually receive an aborted response; moreover, a correct process receives an aborted response only if some correct process has requested abort.

Correctness

EP1: *Validity:*

Any value decided by a correct process was proposed by the leader of the current or a previous epoch consensus.

The collection of this epoch is either bound or unbound to the decided value.

- If it is unbound, the decided value is proposed by the leader of this round.
- If it is bound, the decided value is has been written by process in a previous epoch. By induction, it is proposed by the leader of a previous round.

Correctness

EP2: *Agreement:*

No two correct processes decide differently.

Immediate from a quorum of Accept messages.

Every two set of $2f+1$ processes intersect in a correct process.

Correctness

EP3: Integrity:

Every correct process decides at most once.

Before issuing decide, the accepted array is nullified, and cannot be populated by a quorum again.

Correctness

EP4: *Lock-in:*

If a correct process decides a value in an epoch, no correct process decides a different value in future epochs.

A correct process decides after receiving a quorum (more than $2f$) of Accept messages.

Before sending an Accept message, a correct process stores the decided value v .

In later epochs, the states are collected from a quorum q_2 (more than $2f$).

The quorums q_1 and q_2 intersect in at least one correct process.

Thus. the value v is retrieved and decided in future epochs.

Correctness

EP5: *Termination:*

If the leader is correct, has proposed a value, and no correct process aborts this epoch consensus, then every correct process eventually decides some value.

Progress in State and Collected rounds by the termination of conditional collect.
Progress in Write and Accept rounds as the $2f+1$ correct processes can help each other.

Correctness

EP6: *Abort behavior:*

When a correct process requests abort, it will eventually receive an aborted response; moreover, a correct process receives an aborted response only if some correct process has requested abort.

Immediate from algorithm.